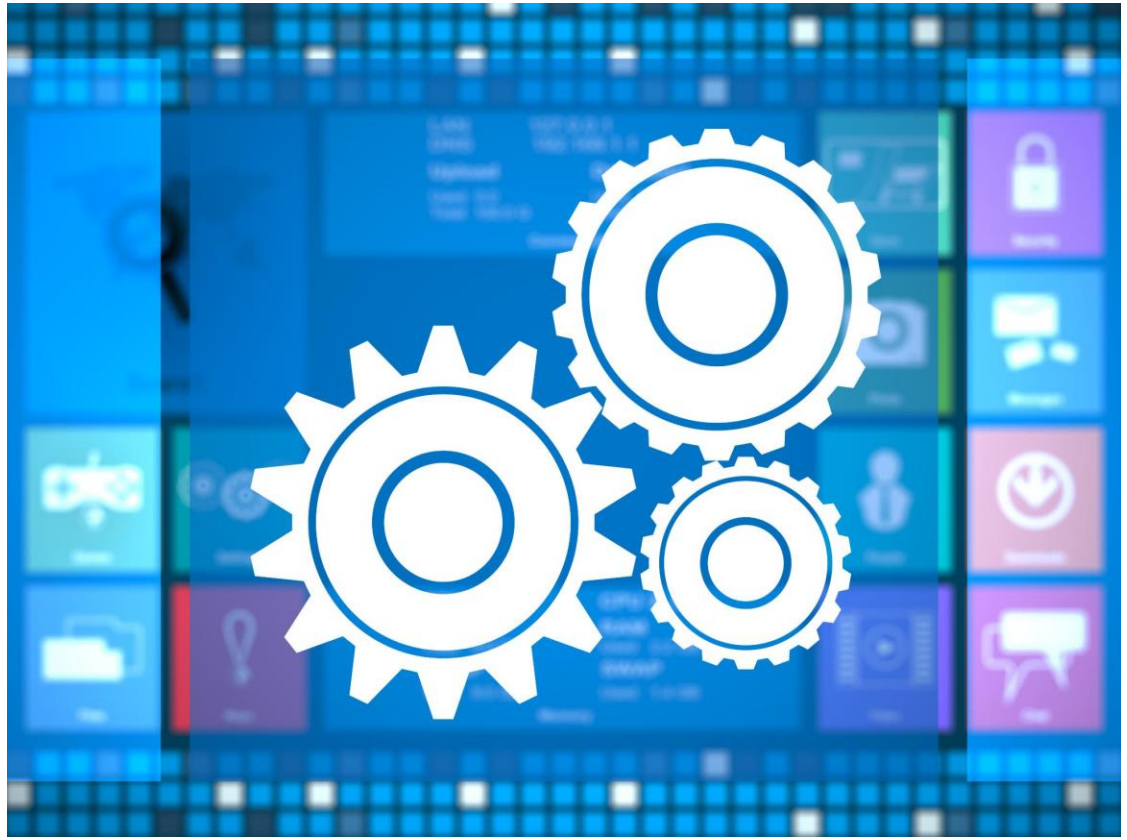
TEST SUMMARY REPORT GENERATOR AGENT – PROOF OF CONCEPT

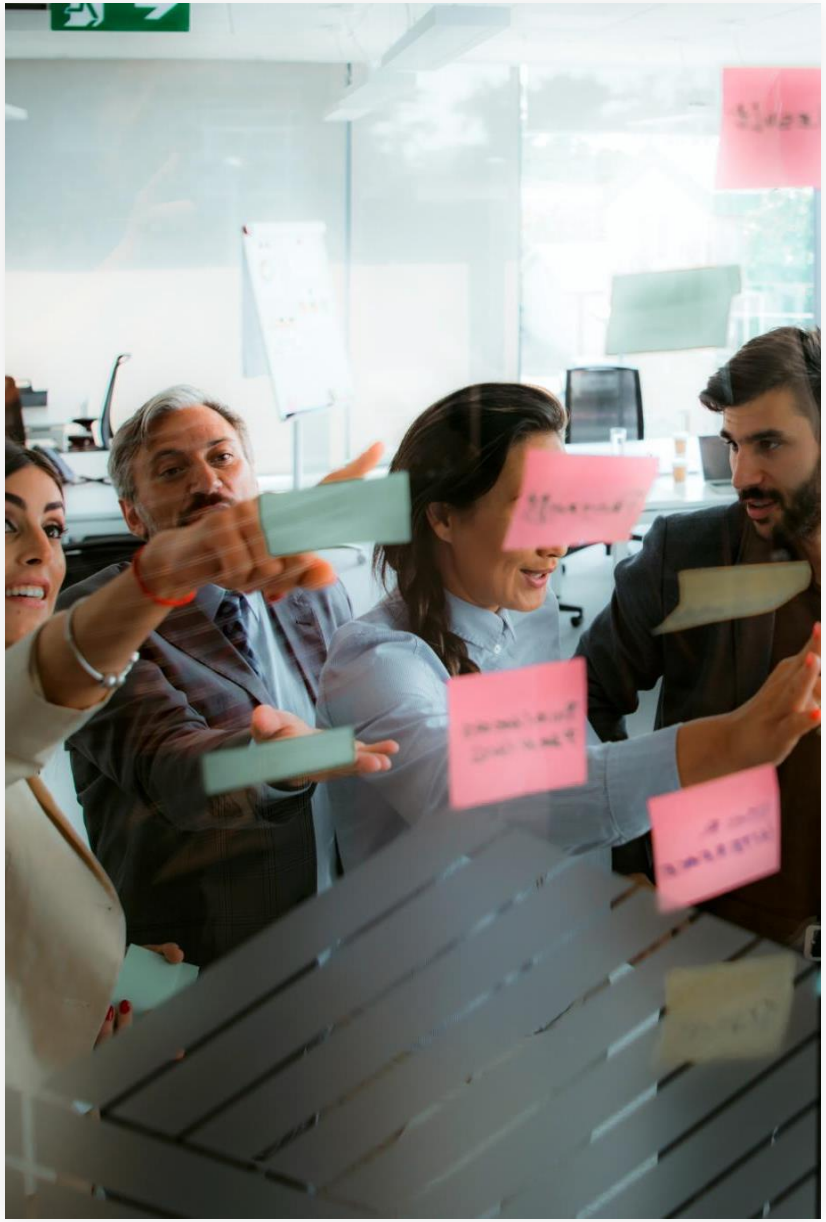
Automated test report creation showcasing innovative solutions

POC OVERVIEW AND CONTEXT

TEST SUMMARY REPORT GENERATOR AGENT – PROOF OF CONCEPT PROPOSAL



- Automation of QA Reporting
  - The PoC automates test summary report creation, boosting efficiency and consistency in QA release documentation.
- Agile and CI/CD Alignment
  - The initiative aligns with Agile practices and continuous integration/delivery for streamlined release reporting.
- Strategic Enablement for QA Engineers
  - Automation enables QA engineers to focus on quality assurance and risk analysis over repetitive tasks.
- Proof of Concept Validation
  - The PoC is structured to validate feasibility, accuracy, and stakeholder acceptance before wider rollout.



BACKGROUND AND PROBLEM STATEMENT

Frequent Agile Releases

Shorter, frequent release cycles increase QA teams' workload for preparing mandatory Test Summary Reports.

Fragmented Test Data

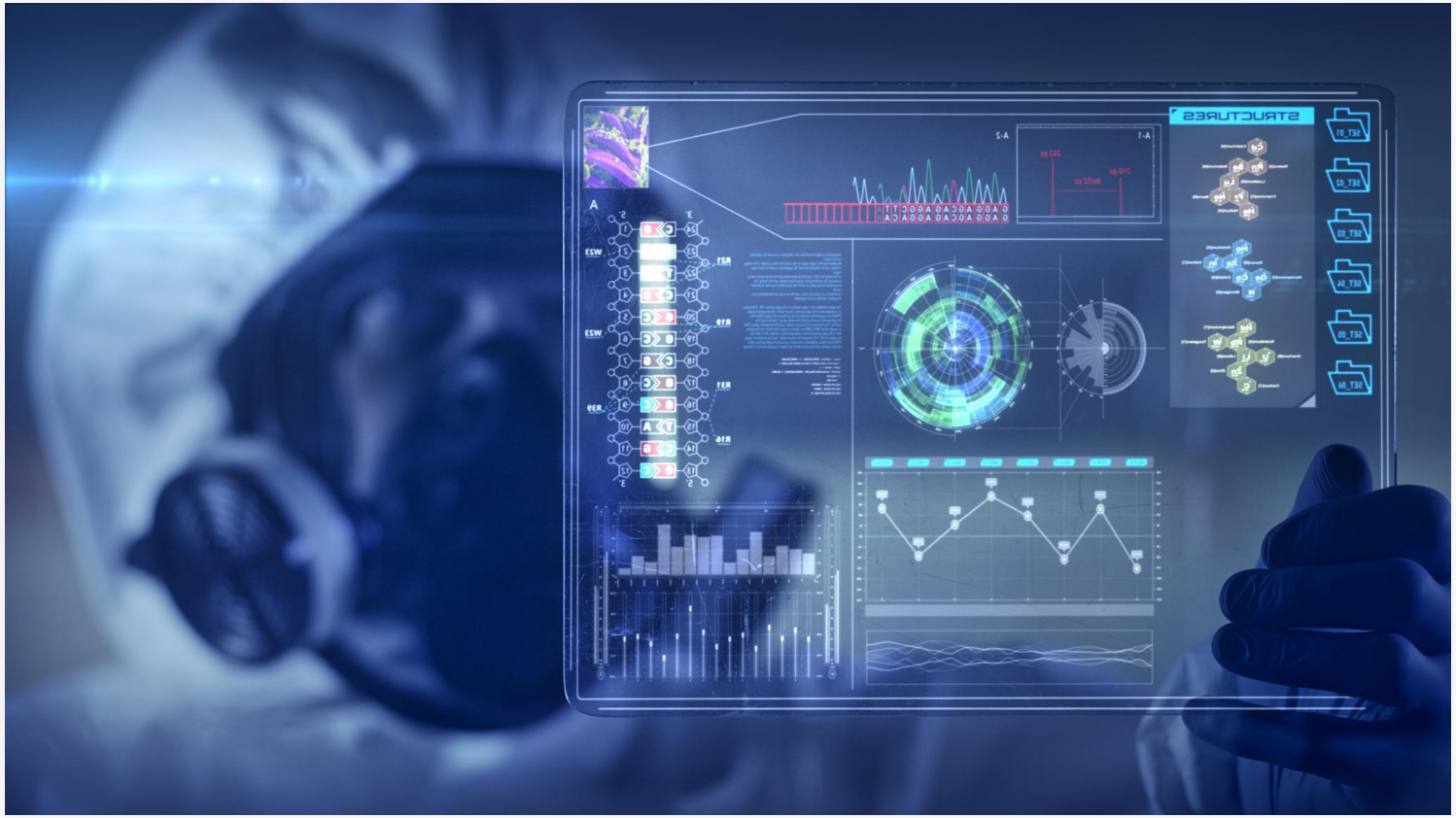
Test data is distributed across JIRA and Zephyr, leading to fragmented information needed for reports.

Manual Reporting Challenges

Manual consolidation and formatting of raw data is time consuming, inconsistent, and hard to scale.

Need for Automation

An intelligent automated reporting solution is needed to synthesize and present test data efficiently.



PROOF OF CONCEPT OBJECTIVE

AI-assisted Report Automation

The PoC aims to automate test summary report generation using AI with minimal manual input.

Data Consolidation from Tools

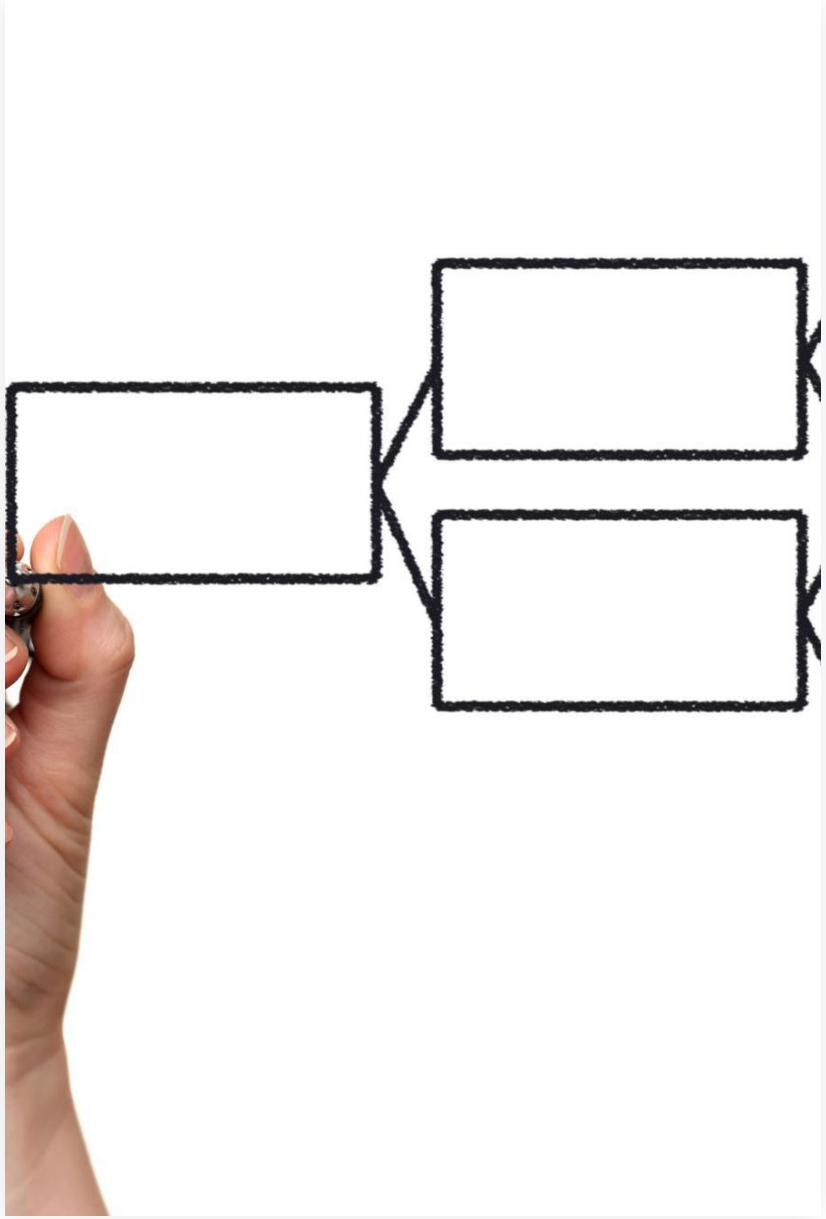
It consolidates release data from JIRA and Zephyr to provide comprehensive test insights.

Contextual Understanding

The system understands relationships between stories, defects, test cases, and outcomes.

Success Measurement

Success is defined by report accuracy, template alignment, and stakeholder acceptance.



PROPOSED SOLUTION OVERVIEW

Orchestration by LangFlow

LangFlow coordinates data retrieval, prompt logic, and output structuring to streamline report generation.

Use of Reference Template

The agent uses approved Test Summary Report templates to maintain consistency in format and language.

Leveraging Large Language Models

Large language models summarize context, interpret results, and create stakeholder-friendly reports.

Repeatable and Configurable Solution

The solution is designed for reuse across projects and releases, ensuring flexibility and scalability.

FUNCTIONAL SCOPE AND CAPABILITIES

Release Scope Identification

The agent queries JIRA to identify all stories and defects linked to the specified Fix Version for release scope.

Test Coverage Mapping

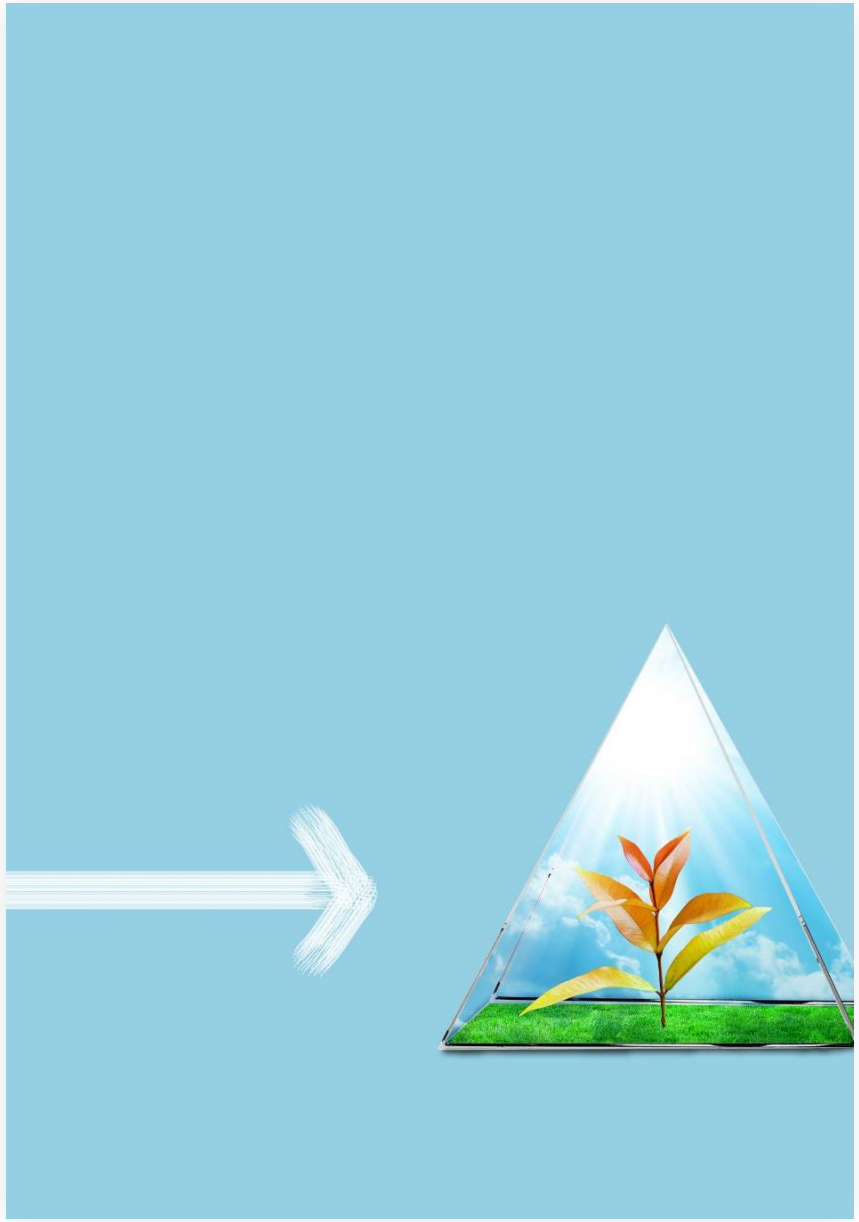
Test cases linked to the release stories and defects are retrieved from Zephyr to map test coverage.

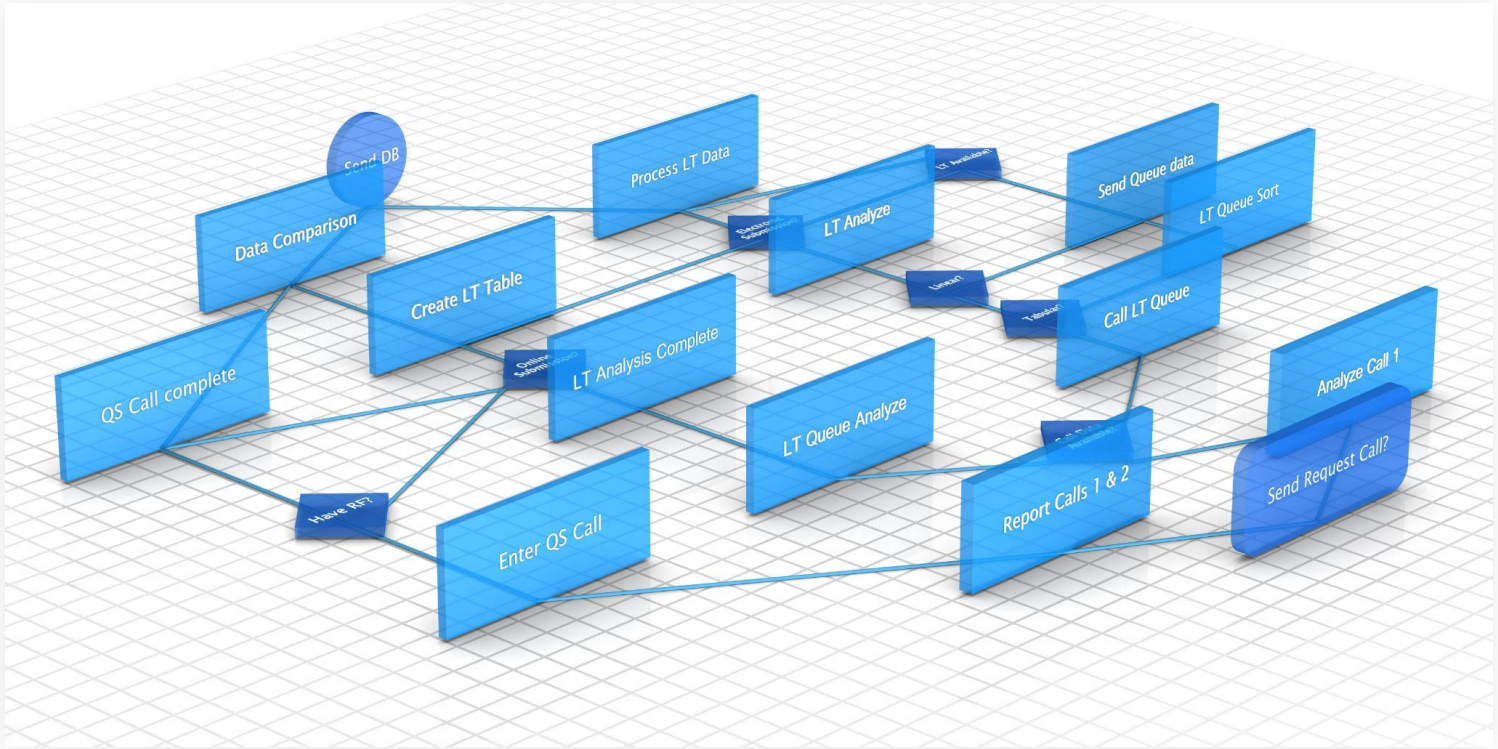
Execution Results Collection

The system collects execution outcomes including pass, fail, and blocked statuses for accurate quality assessment.

Test Summary Report Generation

Collected data is synthesized into a structured Test Summary Report that mirrors existing templates for consistency.





TECHNICAL FLOW AND ARCHITECTURE

User Input and Data Retrieval

User inputs like Project Key and Fix Version drive queries to JIRA and Zephyr to fetch relevant stories, defects, and test cases.

Orchestration by LangFlow

LangFlow manages API interactions and sequences data processing and prompt execution to maintain smooth data flow.

AI-Powered Analysis and Report Generation

A large language model analyzes aggregated data contextually and generates structured narrative content for reports.

Modular Architecture Principles

The architecture emphasizes simplicity, traceability, and extensibility, allowing future enhancements or component replacements.

EXPECTED BENEFITS AND VALUE

Operational Efficiency

Automating report generation reduces manual effort and accelerates QA processes.

Improved Quality and Audit Readiness

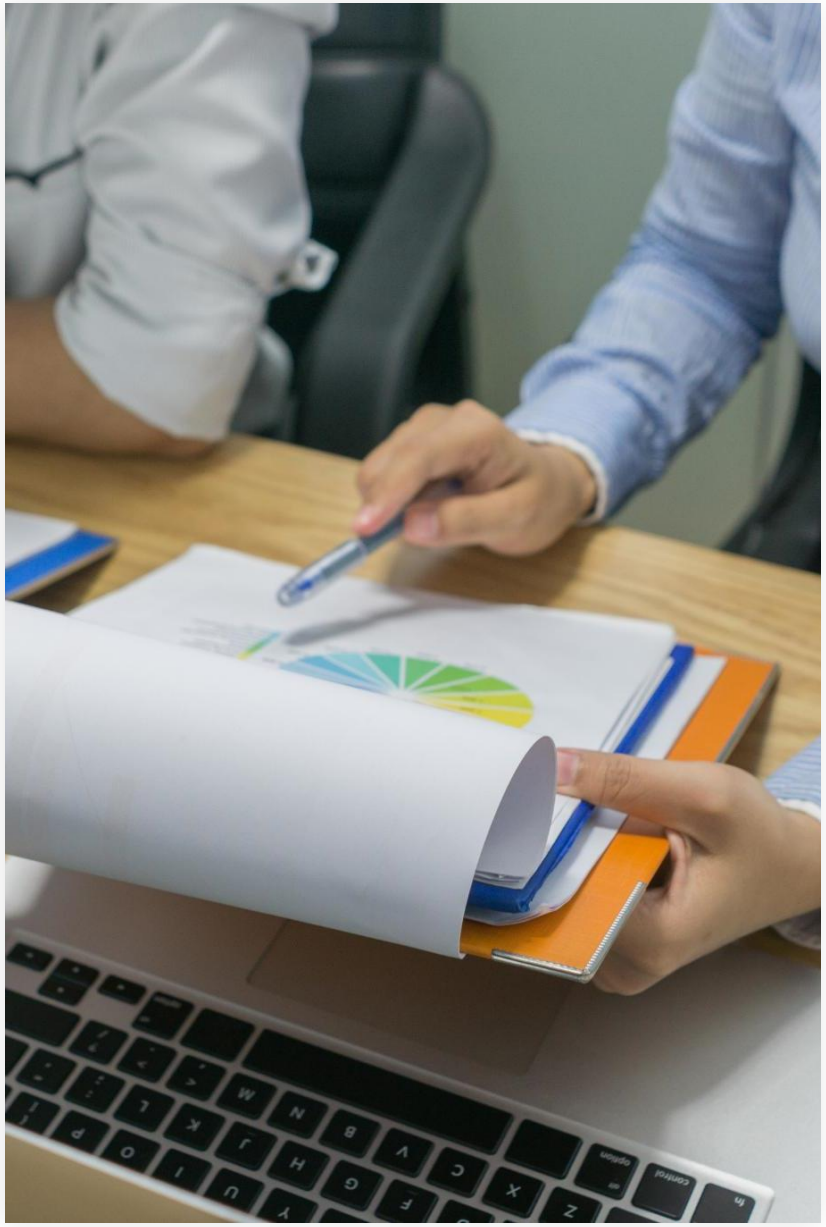
Consistent report structure enhances readability and supports audit compliance.

Enhanced Traceability and Transparency

Automated linkage of stories, defects, and test cases improves release confidence.

Strategic Alignment with Agile DevOps

Solution reduces bottlenecks and supports faster QA sign-offs and UAT transitions.



POC SUCCESS CRITERIA AND NEXT STEPS

Success Measurement

The PoC success is measured by generating accurate Test Summary Reports validated by QA stakeholders.

Additional Success Indicators

Reduced manual effort and positive stakeholder feedback further indicate PoC success and effectiveness.

Next Steps Post-Validation

Next steps include refining prompts, improving error handling, enhancing security, and assessing scalability for rollout.

PoC as a Stepping Stone

The PoC is a foundation towards production readiness, not a standalone experiment, ensuring project continuity.